1 Exercise 4: Arduino MEGA LCD and keypad (Microchip Studio) (MEGA)

1.1 Useful References

- ATmega2560 Datasheet
- AVR LIBC manual

1.2 Overview

In this task, you need to get the signal from the keypad as an input, which will be presented as an output on the first line of LCD screen. All the numbers/letters on the keypad should be able to be shown on the LCD screen. Additionally when numbers 0 to 9 are entered they should be accumulated in an `uint32_t` variable memory and printed on the second line of LCD. When key * is pressed the memory variable should be zeroed. Make sure that memory variable is protected from integer overflow.

1.3 Building Circuit

Build the circuit using these components:

- Arduino Mega (Mega 2560)
- LCD display
- Keypad
- Potentiometer (10 kOhm)
- 1 x 220 Ohm resistor
- Wires

The schematic of the circuit is as follows.

- Make the connection between the keypad and Arduino MEGA, by using file `mega_keypad_model.pdf` as a guide (found in the Keypad library for MEGA folder)
- Connect LCD D4 – D7 pins to MEGA pins 3 – 6
- Connect LCD RS, RW and E pins to MEGA pins 9 - 11
- LCD V0 pin is connected to the middle leg of potentiometer. 2 other legs of potentiometer should be connected to power and ground (order does not matter).
- LCD VDD is connected to 5V bar
- LCD VSS and K are connected to ground
- LCD A pin is connected to 5V bus via 220 Ohm current limiting resistor
- Connect any of 5V pins of Mega board to your circuit's 5V bar.
- Connect any ground pin of Mega board to your circuit's ground bar.

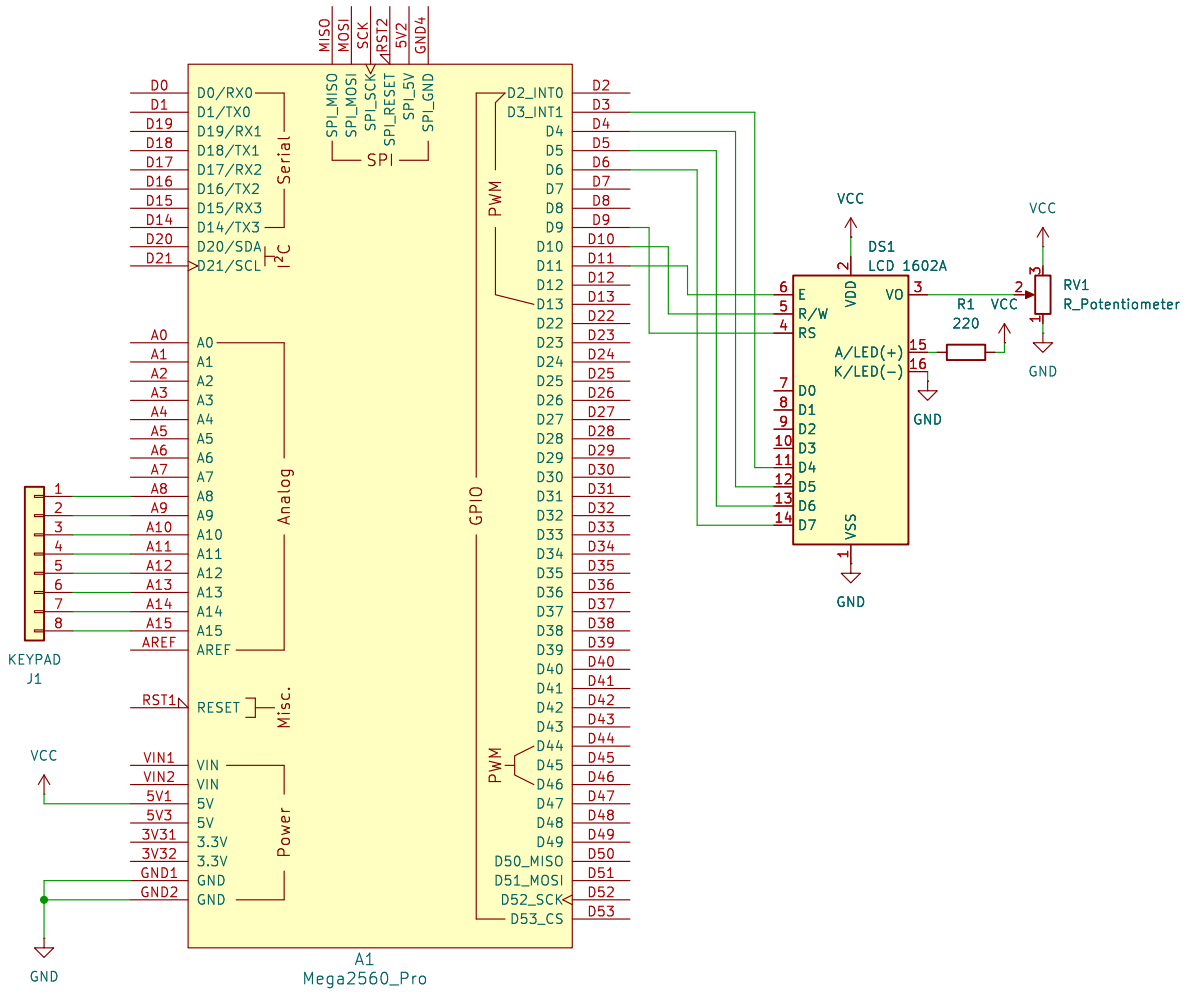

Figure 1: Circuit schematic.

1.4 Included Code

Create main.c file in your project. Include the following provided headers and libraries in your project and add the required `#include` directives to the main.c unit:

- mcu.h for microcontroller-related defines
- uart.h and uart.c for serial port driver implemented in Exercise 1
- delay.h, stdutils.h, keypad.c and keypad.h for Keypad driver
- lcd.c and lcd.h for LCD driver

Additionaly include library headers

```
#include <util/delay.h>
#include <string.h>
#include <stdio.h>
#include <inttypes.h>
```

1.5 LCD write helper

First, write a helper function that will output strings on LCD screen in safe manner and log the copy to serial port output for debugging.

```
#define LCD_MAX_STRING (32)

/**
 * Protected write to LCD that checks that provided pointer is a valid
 * null-terminated string.
 * @param string Pointer to the string that should be printed
 */
static void write_to_lcd(const char *string) {
        uint8_t len = strnlen(string, LCD_MAX_STRING);
        if (LCD_MAX_STRING == len) {
                printf("Failed to print LCD string. Too big or lacks NULL-terminator.\r\n");
                // Since we have null-terminator we print the bad string one character at a time.
                for (uint8_t i = 0; i < len; i++)
                {
                    printf("%c", string[i]);
                }
                printf("\r\n");
                handle_error(1);
        } else {
            printf("LCD output: '%s'\r\n", string);
            lcd_puts(string);
        }
}
```

Here safe string length checker strnlen to check if the variable points to a null-terminated string of no longer than `LCD_MAX_STRING` characters (including null-terminator). If the input is invalid the error message is printed to serial with debug info and the program goes to frozen state. If the string is safe to print the output event is logged to the serial port and printed to the LCD.

1.6 Initialization

Write `handle_error` function similar to previous exercises, then write a `void setup(void)` function that inplements the following initialization steps.

- Initialize serial port for standard library
- Initialize the keypad using the `KEYPAD_Init()`.

- Initialize LCD by calling `lcd_init(LCD_DISP_ON);`
- Clear LCD screen with `lcd_clrscr();`
- Write 'Ready' to LCD with `write_to_lcd` inplemented before

Log progress of initialization to serial port starting right after calling `setup_uart_io();` and checking its exit code.

1.7 Main function and loop

Start main function by declaring a buffer for formatting strings and memory variable.

```
int main(void) {
        static char key_str[32];
        setup();

        uint32_t memory = 0;

        while (1) {
```

In the while loop:

- Get the signal from the keypad, you can use the `KEYPAD_GetKey()` function. Note this call is blocking meaning that your program will not continue until some button press is registered. In case of circuit issues this might look like a frozen program. Therefore use logging to serial around this call to make sure hw issues can be diagnosed correctly.
- Use `uint8_t key_value = key_signal - '0';` to convert the ASCII returnes by the keypad library to numeric value.
- If value is a number multiply memory by 10 and add the new number. Check for integer overflows.
- If key '*' is pressed reset memory value to 0.
- Use `snprintf` to format output strings for both LCD lines.

  ```
  snprintf(key_str, sizeof(key_str), "Key Pressed: %c", key_signal);
  snprintf(key_str, sizeof(key_str), "Mem: %"PRIu32"", memory);
  ```

- Use `lcd_clrscr()` to clear the LCD screen.
- Use `write_to_lcd()` to present the output on the LCD screen.